ITA0615 – MACHINE LEARNING
CO4 AT1

By:
Harshith G
192525394

Annexure A

Short Answer Test

1, Write a Python program to implement the K-Nearest Neighbour (KNN) algorithm for a small dataset. Use Euclidean distance to find nearest neighbors and classify a test instance. Allow the user to input the value of K and display predicted class labels with accuracy.

2. Develop a Python program to implement Locally Weighted Regression. Given a dataset, compute weights based on distance and predict output for a query point. Use a simple kernel function and compare predictions with standard linear regression for better understanding.

3. Write a Python program to implement a simple Radial Basis Function (RBF) network for regression or classification. Use Gaussian functions as activation units and train the model on a small dataset. Display predicted outputs and analyze how centers and widths affect performance.

4. Create a Python program that implements a basic Case-Based Learning system. Store past cases in a dataset and retrieve the most similar case for a new query using distance measures. Output the solution based on the closest match and explain similarity calculation.

5. Write a Python program to implement both KNN and RBF models on the same dataset. Compare their predictions for given test inputs. Analyze differences in computation, accuracy, and output behavior, and display results clearly for better comparison.

1.

Aim: To implement the K-Nearest Neighbour (KNN) classifier using Euclidean distance, take K from the user and report the predicted class and the accuracy.

Concept: KNN is a lazy, instance-based learner. It stores the training data and, for a test instance, finds the K closest training points using the Euclidean distance $d(a, b) = \sqrt{\Sigma(a_i - b_i)^2}$. The class that gets the majority vote among those K neighbours is the prediction.

Program:

```
import math
from collections import Counter


# Training data: [height (cm), weight (kg)] -> T-shirt size
train = [([150, 50], "Small"), ([152, 48], "Small"), ([155, 55], "Small"),
         ([158, 52], "Small"), ([165, 62], "Medium"), ([168, 65], "Medium"),
         ([170, 66], "Medium"), ([172, 68], "Medium"), ([180, 80], "Large"),
         ([183, 85], "Large"), ([185, 88], "Large"), ([188, 90], "Large")]
test = [([154, 50], "Small"), ([159, 56], "Small"), ([166, 63], "Medium"),
        ([171, 67], "Medium"), ([182, 82], "Large"), ([186, 87], "Large")]


def euclidean(a, b):
    return math.sqrt(sum((x - y) ** 2 for x, y in zip(a, b)))


def knn_predict(train, x, k):
    dist = sorted((euclidean(f, x), label) for f, label in train)[:k]
    votes = Counter(label for _, label in dist)
    return votes.most_common(1)[0][0], dist


k = int(input("Enter the value of K: "))
correct = 0
print(f"\n{'Test instance':<14}{'Actual':<9}{'Predicted':<11}Result")
for x, actual in test:
    pred, nn = knn_predict(train, x, k)
    correct += pred == actual
    print(f"{str(x):<14}{actual:<9}{pred:<11}{'Correct' if pred == actual else 'Wrong'}")


print(f"\nNearest neighbours of {test[0][0]} (K={k}):")
for d, label in knn_predict(train, test[0][0], k)[1]:
    print(f"  distance = {d:.3f}  class = {label}")
print(f"\nAccuracy = {correct}/{len(test)} = {correct / len(test) * 100:.2f}%")
```

Output:

```
Enter the value of K: 3
Test instance Actual   Predicted  Result
[154, 50]     Small    Small      Correct
[159, 56]     Small    Small      Correct
[166, 63]     Medium   Medium     Correct
[171, 67]     Medium   Medium     Correct
[182, 82]     Large    Large      Correct
[186, 87]     Large    Large      Correct


Nearest neighbours of [154, 50] (K=3):
  distance = 2.828  class = Small
  distance = 4.000  class = Small
  distance = 4.472  class = Small


Accuracy = 6/6 = 100.00%
```

Explanation / Analysis:

- Distance of the test point to every training point is calculated and sorted; the first K entries are the neighbours.
- The majority label among the K neighbours is the predicted class (ties go to the nearest neighbour).
- With K = 3 all six test instances were classified correctly, so Accuracy = 6/6 = 100%.
- A very small K is sensitive to noise, a very large K blurs the class boundaries; an odd K avoids ties in two-class problems.

2.

Aim: To implement Locally Weighted Regression (LWR) with a Gaussian kernel and compare it with standard linear regression.

Concept:

LWR is a non-parametric method that fits a separate weighted linear model for every query point x_q. Each training point gets a weight $w(i) = \exp(-(x(i) - x_q)^2 / 2\tau^2)$, so nearby points matter more. The parameters are $\theta = (X^T W X)^{-1} X^T W y$, and the prediction is $\theta^T x_q$. τ (bandwidth) controls how local the fit is.

Program:

```
import numpy as np
import matplotlib.pyplot as plt


np.random.seed(1)
X = np.linspace(0, 6, 40)
y = np.sin(X) + 0.15 * np.random.randn(40)        # non-linear data
A = np.c_[np.ones_like(X), X]                     # design matrix [1, x]


def gaussian_kernel(x_query, X, tau):
    return np.exp(-((X - x_query) ** 2) / (2 * tau ** 2))


def lwr_predict(x_query, tau):
    W = np.diag(gaussian_kernel(x_query, X, tau))
    theta = np.linalg.pinv(A.T @ W @ A) @ (A.T @ W @ y)   # (A'WA)^-1 A'Wy
    return np.array([1, x_query]) @ theta


theta_lr = np.linalg.pinv(A.T @ A) @ A.T @ y      # standard linear regression
lr_predict = lambda x: np.array([1, x]) @ theta_lr


tau = 0.5
print(f"Kernel: Gaussian, tau = {tau}")
print(f"\n{'x':>5}{'True sin(x)':>13}{'LWR':>9}{'Linear Reg':>12}")
for xq in [0.5, 1.5, 3.0, 4.5, 5.5]:
    print(f"{xq:>5.1f}{np.sin(xq):>13.3f}{lwr_predict(xq, tau):>9.3f}{lr_predict(xq):>12.3f}")


mse = lambda p: np.mean((np.sin(X) - p) ** 2)
print(f"\nMSE vs true curve  -> LWR: {mse([lwr_predict(x, tau) for x in X]):.4f}"
      f" | Linear Regression: {mse([lr_predict(x) for x in X]):.4f}")
print("\nEffect of bandwidth tau on LWR (MSE):")
for t in [0.1, 0.3, 0.5, 1.0, 5.0]:
    print(f"  tau = {t:<4} MSE = {mse([lwr_predict(x, t) for x in X]):.4f}")


xs = np.linspace(0, 6, 200)
plt.figure(figsize=(6, 3.4))
plt.scatter(X, y, s=14, c="gray", label="Data")
plt.plot(xs, [lwr_predict(x, tau) for x in xs], "r", label="LWR (tau=0.5)")
plt.plot(xs, [lr_predict(x) for x in xs], "b--", label="Linear regression")
plt.legend(); plt.xlabel("x"); plt.ylabel("y"); plt.title("LWR vs Linear Regression")
plt.tight_layout(); plt.savefig("q2.png", dpi=150); plt.show()
```

Output:

```
Kernel: Gaussian, tau = 0.5


    x  True sin(x)      LWR  Linear Reg
  0.5       0.479    0.421       0.785
  1.5       0.997    0.859       0.467
  3.0       0.141    0.135      -0.010
```

```
4.5     -0.978  -0.912     -0.488
5.5     -0.706  -0.658     -0.806


MSE vs true curve  -> LWR: 0.0075 | Linear Regression: 0.1871


Effect of bandwidth tau on LWR (MSE):
  tau = 0.1  MSE = 0.0081
  tau = 0.3  MSE = 0.0040
  tau = 0.5  MSE = 0.0075
  tau = 1.0  MSE = 0.0461
  tau = 5.0  MSE = 0.1842
```

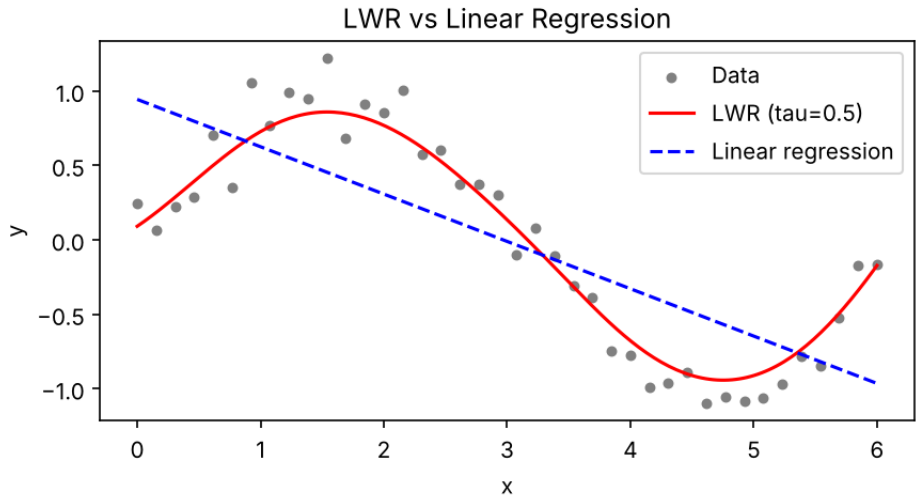


Figure: output plot for Question 2

Explanation / Analysis:

- The data follow a sine curve, which a single straight line cannot represent. Linear regression gives MSE = 0.1871, whereas LWR ($\tau = 0.5$) gives MSE = 0.0075.
- LWR predictions (e.g. 0.859 at x = 1.5) stay close to the true values, while linear regression (0.467) misses the curve.
- Small τ follows the noise (over-fitting), very large τ makes all weights equal and LWR becomes ordinary linear regression (MSE 0.1842 at $\tau = 5$). A moderate τ (0.3 – 0.5) is best.
- Drawback: the model is refitted for every query, so prediction is slower than linear regression.

3.

Aim: To implement a Radial Basis Function (RBF) network with Gaussian hidden units for regression and study the effect of centres and widths.

Concept:
An RBF network has an input layer, one hidden layer of Gaussian units $\varphi_j(x) = \exp(-(x - c_j)^2 / 2\sigma^2)$ and a linear output layer. The centres c_j and width σ are fixed first; the output weights are then found by linear least squares, so training is fast and needs no back-propagation.

Program:

```
import numpy as np
import matplotlib.pyplot as plt


np.random.seed(0)
X = np.linspace(0, 2 * np.pi, 30)
y = np.sin(X) + 0.05 * np.random.randn(30)
X_test = np.linspace(0, 2 * np.pi, 100)


def hidden(x, centers, width):                  # Gaussian activation units
    return np.exp(-(x[:, None] - centers[None, :]) ** 2 / (2 * width ** 2))


def train_rbf(X, y, centers, width):
    P = np.c_[hidden(X, centers, width), np.ones(len(X))]     # + bias
    return np.linalg.lstsq(P, y, rcond=None)[0]               # output weights


def predict(x, centers, width, w):
    return np.c_[hidden(x, centers, width), np.ones(len(x))] @ w


def test_mse(n, width):
    c = np.linspace(0, 2 * np.pi, n)
    w = train_rbf(X, y, c, width)
    return np.mean((predict(X_test, c, width, w) - np.sin(X_test)) ** 2)


centers, width = np.linspace(0, 2 * np.pi, 6), 1.0
w = train_rbf(X, y, centers, width)
print("RBF network: 6 Gaussian centres, width = 1.0")
print("Output weights:", np.round(w, 3))
print(f"\n{'x':>6}{'Actual sin(x)':>15}{'RBF output':>12}")
for xq in [0.5, 1.5, 3.0, 4.5, 6.0]:
    print(f"{xq:>6.1f}{np.sin(xq):>15.3f}{predict(np.array([xq]), centers, width, w)[0]:>12.3f}")


print("\nTest MSE for different number of centres (rows) and widths (columns)")
widths = [0.1, 0.5, 1.0, 3.0]
print(f"{'centres':>8}" + "".join(f"{'w=' + str(s):>10}" for s in widths))
for n in [2, 4, 6, 10]:
    print(f"{n:>8}" + "".join(f"{test_mse(n, s):>10.4f}" for s in widths))


plt.figure(figsize=(6, 3.4))
plt.scatter(X, y, s=14, c="gray", label="Training data")
for n, s, st in [(6, 1.0, "r"), (10, 0.1, "g--"), (6, 3.0, "b:")]:
    c = np.linspace(0, 2 * np.pi, n); ww = train_rbf(X, y, c, s)
    plt.plot(X_test, predict(X_test, c, s, ww), st, label=f"{n} centres, width {s}")
plt.legend(fontsize=7); plt.title("Effect of centres and width"); plt.tight_layout()
plt.savefig("q3.png", dpi=150); plt.show()
```

Output:

```
RBF network: 6 Gaussian centres, width = 1.0
Output weights: [-0.338  0.982  0.337 -0.307 -1.087  0.485  0.017]


   x  Actual sin(x)  RBF output
 0.5         0.479       0.499
 1.5         0.997       1.037
 3.0         0.141       0.162
 4.5        -0.978      -1.014
 6.0        -0.279      -0.219


Test MSE for different number of centres (rows) and widths (columns)
 centres     w=0.1     w=0.5     w=1.0     w=3.0
       2    0.4954    0.4612    0.3161    0.1588
       4    0.4152    0.1347    0.0149    0.0013
       6    0.3599    0.0109    0.0014    0.0010
      10    0.2582    0.0017    0.0013    0.0013
```

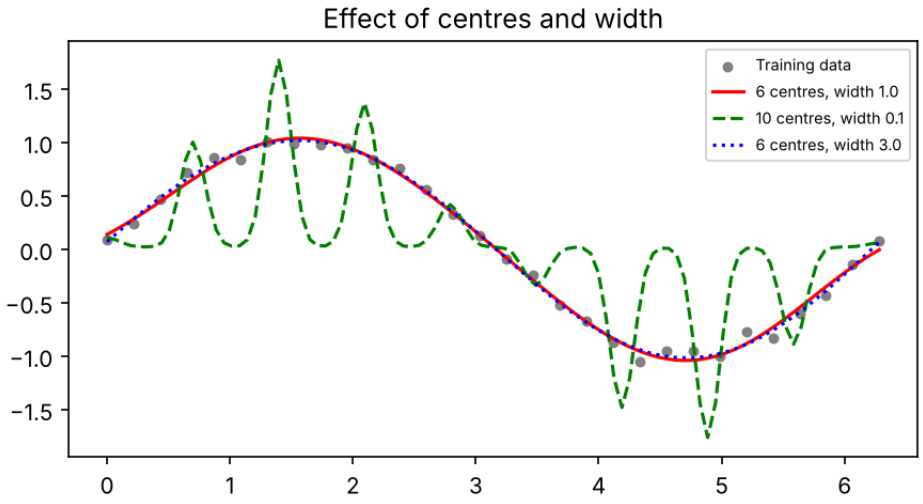


Figure: output plot for Question 3

Explanation / Analysis:

- Six Gaussian centres with width 1.0 approximate sin(x) closely (e.g. 1.037 against 0.997 at x = 1.5).
- Centres: too few centres (2) cannot capture the curve (MSE 0.32 at σ = 1.0); more centres reduce the error (0.0013 with 10 centres).
- Width: a very small width (0.1) makes each unit respond to a tiny region, leaving gaps between centres (MSE 0.36 with 6 centres); a wider σ gives smooth coverage. Too wide a σ with very few centres makes the units overlap and lose local detail.
- Good performance needs enough centres and a width matched to the spacing between centres.

4.

Aim: To build a basic Case-Based Reasoning (CBR) system that retrieves the most similar past case and reuses its solution.

Concept:

CBR solves a new problem by remembering a similar old one. The cycle is Retrieve – Reuse – Revise – Retain. Features are first min-max normalised so that no feature dominates. Similarity is computed from the Euclidean distance d as Similarity = 1 / (1 + d); a smaller distance means a higher similarity (1 means identical).

Program:

```
import numpy as np


features = ["Temp(C)", "Humidity(%)", "Rainfall(mm)", "Soil pH"]
case_base = [  # (problem description, solution)
    ([27, 80, 230, 6.0], "Rice"),   ([25, 82, 200, 5.8], "Rice"),
    ([28, 85, 250, 6.2], "Rice"),   ([18, 55, 80, 6.8], "Wheat"),
    ([15, 60, 90, 7.0], "Wheat"),   ([20, 50, 70, 6.5], "Wheat"),
    ([30, 60, 100, 7.2], "Cotton"), ([32, 55, 90, 7.5], "Cotton"),
    ([24, 65, 110, 6.5], "Maize"),  ([22, 70, 120, 6.3], "Maize")]


def normalise(v):                    # min-max scaling using case-base ranges
    data = np.array([c[0] for c in case_base], float)
    lo, hi = data.min(0), data.max(0)
    return (np.array(v, float) - lo) / (hi - lo)


def retrieve(query):
    q = normalise(query)
    scored = []
    for i, (prob, sol) in enumerate(case_base):
        d = np.sqrt(np.sum((q - normalise(prob)) ** 2))   # Euclidean distance
        scored.append((1 / (1 + d), d, i, sol))           # similarity = 1/(1+d)
    return sorted(scored, reverse=True)


for query in ([26, 81, 220, 6.1], [19, 58, 85, 6.9]):
    print("=" * 60)
    print("New problem (query):", dict(zip(features, query)))
    ranked = retrieve(query)
    print(f"{'Case':>5}{'Solution':>10}{'Distance':>10}{'Similarity':>12}")
    for sim, d, i, sol in ranked[:4]:
        print(f"{i + 1:>5}{sol:>10}{d:>10.4f}{sim:>12.4f}")
    sim, d, i, sol = ranked[0]
    print(f"\nMost similar case: Case {i + 1}  {case_base[i][0]}")
    q, c = normalise(query), normalise(case_base[i][0])
    for f, a, b in zip(features, q, c):
        print(f"  {f:<13} (q - c)^2 = ({a:.3f} - {b:.3f})^2 = {(a - b) ** 2:.4f}")
    print(f"  distance = sqrt(sum) = {d:.4f},  similarity = 1/(1+{d:.4f}) = {sim:.4f}")
    print(f"==> Suggested solution (reuse): {sol}")
    case_base.append((query, sol))      # retain the solved case for future use
print("\nCase base size after retaining new cases:", len(case_base))
```

Output:

```
============================================================
New problem (query): {'Temp(C)': 26, 'Humidity(%)': 81, 'Rainfall(mm)': 220, 'Soil pH': 6.1}
 Case  Solution  Distance  Similarity
    1      Rice    0.1040      0.9058
    2      Rice    0.2185      0.8206
    3      Rice    0.2411      0.8057
```

```
  10     Maize    0.6904      0.5916


Most similar case: Case 1  [27, 80, 230, 6.0]
  Temp(C)       (q - c)^2 = (0.647 - 0.706)^2 = 0.0035
  Humidity(%)   (q - c)^2 = (0.886 - 0.857)^2 = 0.0008
  Rainfall(mm)  (q - c)^2 = (0.833 - 0.889)^2 = 0.0031
  Soil pH       (q - c)^2 = (0.176 - 0.118)^2 = 0.0035
  distance = sqrt(sum) = 0.1040,  similarity = 1/(1+0.1040) = 0.9058
==> Suggested solution (reuse): Rice
============================================================
New problem (query): {'Temp(C)': 19, 'Humidity(%)': 58, 'Rainfall(mm)': 85, 'Soil pH': 6.9}
 Case  Solution  Distance  Similarity
    4     Wheat    0.1226      0.8908
    5     Wheat    0.2507      0.7995
    6     Wheat    0.3435      0.7443
    9     Maize    0.4485      0.6904


Most similar case: Case 4  [18, 55, 80, 6.8]
  Temp(C)       (q - c)^2 = (0.235 - 0.176)^2 = 0.0035
  Humidity(%)   (q - c)^2 = (0.229 - 0.143)^2 = 0.0073
  Rainfall(mm)  (q - c)^2 = (0.083 - 0.056)^2 = 0.0008
  Soil pH       (q - c)^2 = (0.647 - 0.588)^2 = 0.0035
  distance = sqrt(sum) = 0.1226,  similarity = 1/(1+0.1226) = 0.8908
==> Suggested solution (reuse): Wheat


Case base size after retaining new cases: 12
```

Explanation / Analysis:

- Case base: 10 crop cases described by temperature, humidity, rainfall and soil pH, each with a solution (crop).
- For the first query (26, 81, 220, 6.1) the closest case is Case 1 with distance 0.1040, so similarity = 1/(1 + 0.1040) = 0.9058 and the suggested solution is Rice.
- The squared normalised differences of the four features are added, the square root gives the distance, and the closest case is reused. The second query is matched to Wheat in the same way.
- Solved queries are retained, so the case base grew from 10 to 12 cases and the system learns over time.

5.

Aim: To implement KNN and an RBF network on the same dataset and compare their predictions, accuracy, computation and output behaviour.

Concept:

KNN is a lazy learner: no training, but each prediction compares the query with every stored point. An RBF network is an eager learner: it spends time on training (k-means centres + least-squares weights) and then predicts quickly using only a few centres. Both can model non-linear boundaries.

Program:

```
import time
import numpy as np


rng = np.random.RandomState(3)
def make_moons(n):
    t = rng.uniform(0, np.pi, n // 2)
    a = np.c_[np.cos(t), np.sin(t)]
    b = np.c_[1 - np.cos(t), 0.5 - np.sin(t)]
    X = np.r_[a, b] + 0.18 * rng.randn(n, 2)
    return X, np.r_[np.zeros(n // 2), np.ones(n // 2)].astype(int)


Xtr, ytr = make_moons(300)
Xte, yte = make_moons(100)


# ---------------- KNN (lazy learner) ----------------
def knn_predict(X, k=5):
    d = np.linalg.norm(X[:, None, :] - Xtr[None, :, :], axis=2)
    idx = np.argsort(d, axis=1)[:, :k]
    votes = ytr[idx].mean(axis=1)                # fraction of class-1 neighbours
    return (votes >= 0.5).astype(int), votes


# ---------------- RBF network (eager learner) ----------------
def kmeans_centers(X, m, iters=20):
    C = X[rng.choice(len(X), m, replace=False)]
    for _ in range(iters):
        lab = np.argmin(np.linalg.norm(X[:, None] - C[None], axis=2), axis=1)
        C = np.array([X[lab == j].mean(0) if np.any(lab == j) else C[j] for j in range(m)])
    return C


def rbf_layer(X, C, s):
    return np.exp(-np.sum((X[:, None] - C[None]) ** 2, axis=2) / (2 * s ** 2))


t0 = time.perf_counter()
m = 10
C = kmeans_centers(Xtr, m)
dmax = max(np.linalg.norm(a - b) for a in C for b in C)
s = dmax / np.sqrt(2 * m)                        # common width heuristic
P = np.c_[rbf_layer(Xtr, C, s), np.ones(len(Xtr))]
W = np.linalg.lstsq(P, np.eye(2)[ytr], rcond=None)[0]
rbf_train_time = time.perf_counter() - t0


def rbf_predict(X):
    out = np.c_[rbf_layer(X, C, s), np.ones(len(X))] @ W
    return out.argmax(1), out


t0 = time.perf_counter(); pk, vk = knn_predict(Xte); knn_time = time.perf_counter() - t0
t0 = time.perf_counter(); pr, outr = rbf_predict(Xte); rbf_time = time.perf_counter() - t0
```

```
print("Dataset: two interleaving half-moons | train = 300, test = 100")
print(f"RBF: {m} centres, width = {s:.3f}\n")
print(f"{'Test input':<20}{'Actual':>7}{'KNN(k=5)':>10}{'P(class1)':>11}{'RBF':>6}{'RBF scores':>20}")
for i in [0, 1, 2, 50, 51, 52]:
    print(f"{str(np.round(Xte[i], 2)):<20}{yte[i]:>7}{pk[i]:>10}{vk[i]:>11.2f}{pr[i]:>6}"
          f"{str(np.round(outr[i], 2)):>20}")
print("\nComparison")
print(f"{'Model':<8}{'Accuracy':>10}{'Training time':>20}{'Prediction time':>20}")
print(f"{'KNN':<8}{(pk == yte).mean() * 100:>9.1f}%{'~0 ms (stores data)':>20}{knn_time * 1000:>17.2f} ms")
print(f"{'RBF':<8}{(pr == yte).mean() * 100:>9.1f}%{rbf_train_time * 1000:>17.2f} ms{rbf_time * 1000:>17.2f} ms")
print(f"\nKNN stores {len(Xtr)} points; RBF stores only {m} centres + {m + 1} x 2 weights.")
```

Output:

```
Dataset: two interleaving half-moons | train = 300, test = 100
RBF: 10 centres, width = 0.631


Test input          Actual  KNN(k=5)  P(class1)   RBF         RBF scores
[-0.31  0.63]          0       0       0.20     0          [0.72 0.28]
[0.18 1.06]            0       0       0.00     0           [ 1. -0.]
[0.89 0.69]            0       0       0.00     0         [ 1.07 -0.07]
[ 1.53 -0.6 ]          1       1       1.00     1         [-0.07  1.07]
[ 0.77 -0.14]          1       0       0.40     1          [0.39 0.61]
[ 0.08 -0.28]          1       1       1.00     1         [-0.14  1.14]


Comparison
Model     Accuracy       Training time     Prediction time
KNN         98.0% ~0 ms (stores data)           1.78 ms
RBF         98.0%            3.89 ms            0.15 ms


KNN stores 300 points; RBF stores only 10 centres + 11 x 2 weights.
```

Explanation / Analysis:

- Dataset: two interleaving half-moons (300 training and 100 test points). KNN uses K = 5; the RBF network uses 10 centres.
- Accuracy: both models reach 98% on the test set. They differ on a boundary point such as (0.77, −0.14): KNN gives 0.40 votes for class 1 and predicts 0, whereas the RBF scores (0.39, 0.61) predict 1.
- Computation: KNN needs no training but stores all 300 points and is slower at prediction (≈1.8 ms for 100 queries); RBF needs ≈3.9 ms training and then predicts in ≈0.15 ms using only 10 centres.
- Output behaviour: KNN gives a vote fraction (a step-like, locally jagged boundary); the RBF output is a smooth continuous score for each class. Timing values vary slightly from run to run.